QueryAPIGate DOCUMENTATION · SECURITY

Architecture overview

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=ARCHITECTURE

For enterprise architects and security reviewers. Describes release **0.34.2**.

This document covers how QueryAPIGate is built and where its controls sit. The reasons behind each control, and where each one stops, are in THREAT_MODEL.md. What is kept, where and for how long is in DATA_PROTECTION.md. The hardening checklist and vulnerability reporting are in SECURITY.md. Production setup is in DEPLOYMENT.md. Answers to common questionnaire items are in QUESTIONNAIRE.md.

1. What it is, and the trust boundary

QueryAPIGate is a self-hosted gateway, open source under Apache-2.0. It turns saved SQL queries into REST endpoints, MCP tools for AI agents, live server-sent events and exports. A saved query is versioned, and its parameters are typed and bound to the driver rather than written into the SQL. Every call goes through one governed path: authentication, limits, grants, a SQL guard, masking and a run-history record. It is a Python (Flask) application. The admin UI, the Console, is a React single-page app that ships inside the same package.

Trust boundary. QueryAPIGate runs inside the customer's network, on infrastructure the customer operates. No part of it is a hosted service. It connects outward only to systems the deployer configures:

- the databases and file locations that connections name;
- its own metadata store (SQLite or PostgreSQL) and, optionally, Redis;
- an identity provider, for Console sign-in, signed-in app users and MCP clients;
- secret providers: files, HashiCorp Vault, and AWS (experimental);
- export destinations (object-storage buckets or allowed local folders);
- a SIEM, an OpenTelemetry collector and a Prometheus scraper.

QueryAPIGate does not terminate TLS. A reverse proxy or ingress in front of it does that, and is required once any key exists. QueryAPIGate is not a network perimeter or a web application firewall: network policy, a VPN or a proxy still decides who can reach it at all.

2. Components

All components come from one image (`ghcr.io/anantharajuc/queryapigate`) and one code base. They share the metadata store.

COMPONENT	PROCESS AND PORT	WHAT IT SERVES
Main server	`queryapigate serve`, or gunicorn in the image; `:5000` (`QUERYAPIGATE_PORT`)	Runtime routes (`/q/<name>`, `/execute_sql`, `/catalog`, `/connections/<name>/schema`), `GET /events` for a few streams, the Management API, the Console, `/docs`, `/openapi.json`, `/metrics`, `/health`
Management API	On the main server, at `/api/v1`	Versioned, resource-oriented administration: connections, saved queries and versions, keys, roles, collections, history, audit log, settings, exports. Administrators only, checked by capability per role.
Console	On the main server, at `/console`	Static React build, a client of the public HTTP APIs only. It bundles everything it runs.
MCP server	`queryapigate mcp`; `:5001` (`QUERYAPIGATE_MCP_PORT`), at `/mcp`	Saved queries as tools for AI agents. It uses the same authentication and grant checks as REST. Results are capped at `QUERYAPIGATE_MCP_MAX_ROWS`.
Events server	`queryapigate events`; `:5002` (`QUERYAPIGATE_EVENTS_PORT`)	Server-sent events for many clients, read from the store's event log, plus its own `/metrics` and `/health`
Export CLI	`queryapigate export` (local file), `queryapigate exports run` (saved exports to a destination)	Runs a saved query to a file or bucket. Scheduling comes from outside, such as host cron or a Kubernetes CronJob.
Other CLI	`queryapigate init`, `backup`, `migrate-to-postgres`, `rekey`, `audit verify`, `history forget`, `admins ...`	Operations on the store. Anyone with a shell and store access is outside the application's access model.

The image runs as a non-root user (UID 1000), with one gunicorn worker and eight threads per container. To add capacity, add containers.

3. Diagram

```text
CALLERS                    FRONT DOORS (inside the customer network)

Apps, partners --+   +------------------------------+
Browsers         |   | Reverse proxy / ingress      |
(Console, SSE)   +-->| TLS is terminated here       |
AI agents (MCP) -+   +----+----------+----------+---+
                          |          |          |
cron / CronJob            v          v          v
      |              +---------+ +--------+ +--------+
      v              | main    | | MCP    | | events |
+------------+       | :5000   | | :5001  | | :5002  |
| export CLI |       | REST    | | /mcp   | | SSE    |
+-----+------+       | /api/v1 | |        | |        |
      |              | Console | |        | |        |
      |              +----+----+ +---+----+ +---+----+
      |                   |          |          | steps 1-2 only,
      v                   v          v          | then reads the
+-----------------------------------------+  | event log
| GOVERNED CORE - the same code in each      |  |
| process:                                   |  |
| 1 identify caller  2 apply limits          |  |
| 3 check grant      4 validate parameters   |  |
| 5 guard the SQL    6 run it                |  |
| 7 mask and format  8 record it             |  |
+----+-------------+-------------+-----------+  |
     |             |             |              |
     v             v             v              v
Your databases  Files through  Export        +----------------------------+
PostgreSQL,     DuckDB, only   destinations  | Store, used by every       |
MySQL, SQLite,  allowed_paths  (bucket or    | process: SQLite or         |
DuckDB, Click-  (disk, S3,     allowed       | PostgreSQL - config, key   |
House, ...      GCS, R2, web)  folder)       | hashes, history, event     |
                                             | log, audit log             |
                                             +----------------------------+
                                             | Redis - optional; needed   |
                                             | for several instances      |
                                             +----------------------------+

OUTSIDE SYSTEMS
  IdP ............ OIDC (Console), JWKS (app users), OAuth 2.1 (MCP)
  Secrets ........ files, Vault; AWS (experimental)
  SIEM ........... receives the audit log: syslog over TLS, or OTLP logs
  Prometheus ..... scrapes /metrics (bearer token when set)
  OTLP collector . receives traces (experimental)
```

4. The request path

Every front door applies the same governance code (authentication, rate limits, metrics). A saved-query call then passes eight steps. This is a model of what is checked, not a strict order: the server-wide rate limit applies before the key is checked, and a column masking can't trace is refused before the query runs. Codes are from the API.md error catalogue; `code` is under the compatibility promise.

1. **Identify the caller.** Admin key or scoped key (`X-API-Key` ; the admin key compared in constant time, scoped keys looked up by SHA-256 hash), an administrator's token, a JWT (`Authorization: Bearer`), or on MCP an OAuth access token. Fails `401 unauthorized` ; on MCP, a token without the configured scope is `403 insufficient_scope` .
2. **Apply its limits.** Server-wide per-address and per-key rate limits (`429 rate_limited`), usage quotas, checked before the response cache (`429 quota_exceeded`), `max_concurrent` (`429 too_many_concurrent`). `allowed_ips` and `expires_at` fail like a wrong key (`401 unauthorized`), so the caller learns nothing about why.
3. **Check the grant.** The key's `connections` , `collections` or `queries` grant must reach the query or connection: `403 connection_forbidden` or `403 forbidden` ; `403 connection_inactive` ; `404 query_not_found` or `404 not_published` . On the Management API: `403 admin_only` or `403 role_forbidden` .
4. **Validate parameters.** Each value is checked against its declared type and rules, then bound to the driver, never spliced into SQL. A `from_claim` parameter comes from the verified token, never the request. `400 param_required` , `400 param_invalid` , `400 param_from_claim` , `403 sign_in_required` , `403 claim_missing` .
5. **Guard the SQL.** A dialect-aware **regex classifier** enforces one statement (`400 multiple_statements`) and read-only unless writes are allowed, including writes hidden behind a read-only keyword (`403 read_only` , `403 write_op_not_allowed`). A **sqlglot parser** enforces `allowed_tables` (`403 table_not_allowed` ; `403 table_check_failed` when the SQL can't be analysed; `403 table_check_unsupported` on H2 and JDBC). DuckDB file access outside `allowed_paths` is `403 path_not_allowed` .
6. **Run it.** A pooled connection runs the bound statement under a time limit. PostgreSQL, MySQL, ClickHouse and SQLite run read-only queries in read-only sessions; DuckDB, H2 and JDBC are opened read-write, so there the guard is the only lock. A query with `cache_ttl` may be answered from the response cache. `504 query_timeout` , `500 query_failed` , `502 connection_failed` , `503 secret_unavailable` .
7. **Mask and format.** Masking rules on the key, role or export apply to source columns; result columns are traced to them with sqlglot lineage, not matched by name. Anything untraceable is refused, never served unmasked (`403 mask_unresolvable` ; `500 masking_unavailable` without sqlglot). The result is serialised in the requested format (`400 invalid_format`).
8. **Record it.** The run is written to history and the event log in one batch transaction, and metrics are updated. History keeps parameter names, never values. Configuration changes go to the hash-chained audit log with the administrator who made them. If the write queue is full, a run is dropped from both.

A scoped key calling a saved query never receives the database's own error text, which can reveal schema; the server log has it.

5. State and data flows

The full inventory is DATA_PROTECTION.md; a test fails if it misses a store table, a retention setting or an outbound flow.

The store (`queryapigate.db` on the `/data` volume, or PostgreSQL) holds connections (passwords Fernet-encrypted with `QUERYAPIGATE_SECRET_KEY` , or held as references), saved queries and every version's SQL, API keys and admin tokens as SHA-256 hashes only, roles, administrators, SSO sessions by hash, run history, the event log, usage counters, the audit log with its hash chain and SIEM delivery cursors, agents' write requests (30 days), export destinations and watermarks, the instance list, and server-generated keys such as masking's. The admin, Fernet and audit keys are not in the store.

Redis (optional; required for several instances) holds the response cache (masked rows of queries that set `cache_ttl` , for that long), rate-limit counters (the current window), quota counters (40 days) and running-query counts. If Redis fails, the cache misses and rate limits fall back to per-process counting: still enforced, with an alert.

The event log keeps each run as `/events` sends it for `QUERYAPIGATE_EVENTS_RETENTION_HOURS` (default 24). A key sees only its own events, none from before it was created; the admin key and data administrators see all.

Passes through, not kept: result rows (except in the response cache and exports), parameter values, and resolved secrets (memory only, for their lease or `QUERYAPIGATE_SECRET_CACHE_SECONDS`).

Sent outward:

TO	WHAT
Log (stdout), access log	Requests without query strings; SQL or its SHA-256; database error messages, which can quote a value
SIEM	Every audit entry with `seq` and `hash`, by syslog over TLS (mutual TLS optional) or OTLP logs
OTLP collector	Route, query name, rows, caller name and address; never parameter values, rows, statements or credentials
Prometheus	Counts and timings by connection, query and key name
Export destinations	A saved query's rows, after the export's masking
MCP clients	Rows after masking, up to the row cap
Secret providers	Requests for the secrets connections name

`queryapigate history forget` erases a person's runs, events and usage counts; the audit log records the erasure under the caller's SHA-256.

6. Deployment shapes

All three run the same image and code.

	SINGLE INSTANCE	SEVERAL INSTANCES	KUBERNETES (HELM)
Store	SQLite on a volume, or PostgreSQL	PostgreSQL	SQLite (one pod) or PostgreSQL
Redis	Optional	Required	Required for more than one pod
Live events	`GET /events` on the instance	`queryapigate events`, routed by the proxy	Events pods, routed by the ingress
Availability	Restart on failure	Load balancer skips an unhealthy instance	Rolling updates (`maxUnavailable: 0`), PodDisruptionBudget
Tested in CI	Full suite (about 1,300 tests) across Python versions; backup and restore (`test_backup_restore.py`); stores from every release since 0.7 (`test_upgrades.py`); the Compose demo	Three real servers on one store (`test_multinode.py`); previous PyPI release on a store the current code upgraded (`test_rolling_upgrade.py`); full suite on PostgreSQL with Redis 7	`helm lint` (Helm 3 and 4); rendered security contexts, routes and refusals; plain manifests compared; on kind, the PostgreSQL shape installed and upgraded under load with no failed request, and the SQLite shape's data surviving a pod restart

A further CI job runs the whole suite in Red Hat UBI 9 under OpenSSL's FIPS provider.

The chart's pods run non-root (UID 1000), read-only root filesystem, all capabilities dropped, RuntimeDefault seccomp, no service account token; the server writes only to `/data` and `/tmp`. `helm install` refuses more than one pod on SQLite or without Redis, a secret setting in `config`, and any value not in the schema. `/metrics` is token-protected by default; NetworkPolicy is off until the deployer supplies the cluster's rules.

7. Identity integrations

- **Administrators:** four fixed roles (owner, admin, developer, auditor), personal admin tokens stored as hashes with optional expiry. The shared `QUERYAPIGATE_API_KEY` remains a break-glass owner key; once a named owner exists, each use is audited and alerted.
- **Console SSO (OIDC):** authorization code flow with PKCE (S256) as a confidential client, so the browser never holds a token. State, nonce and verifier are server-side, single-use, valid ten minutes; the ID token's signature, issuer, audience, `azp`, nonce and expiry are checked. People are bound by `sub`; roles come from IdP groups at each sign-in. Session cookie `__Host-`, `HttpOnly`, `Secure`, `SameSite=Strict`, stored as a hash, paired with a CSRF token. `QUERYAPIGATE_SSO_ONLY` removes personal tokens from SSO users. Tested against Keycloak 26.
- **App users (JWT/JWKS):** RS256 against a JWKS, or HS256 with a shared secret. `exp` required; `iss` / `aud` enforced (mandatory with a JWKS); HMAC and public-key families can't be mixed; `none` never accepted. Grants come from a role, read live. A JWT can't be revoked before it expires.
- **MCP clients (OAuth 2.1):** the MCP server is a resource server. A token's `aud` must be the MCP server's URL and it must carry the configured scope; the algorithm comes from the provider's published key, never the token header; provider metadata is fetched over `https://` only and must name the configured issuer (RFC 8414). REST and

MCP refuse each other's tokens. A caller's token never reaches a database.

- **Secrets:** `${VAR}`, `${file:...}`, `${vault:...}` and short-lived Vault database credentials (stable since 0.34), each reachable only through deployer allow-lists checked when saved and when used; no reference can reach the server's own settings or credentials. `${aws-sm:...}` and RDS IAM tokens are experimental. The chart supports Vault's Kubernetes auth and IRSA.

8. Security controls summary

CONTROL	WHERE ENFORCED	REFERENCE
Authentication on every request; open-access mode only while no key, administrator or JWT is configured	Request hook in each front door (`governance.py`)	THREAT_MODEL.md
Key and token secrets stored as SHA-256 hashes; rotation with a bounded grace period	Store; `/api/v1/api-keys`	ADR 0009
Scoped grants (connections, collections, queries, writes, tables, IPs, expiry)	`apikeys.py`, before execution	THREAT_MODEL.md
Administrator roles and capabilities; every `/api/v1` operation mapped	`adminroles.py`, before any route runs	ADR 0003
Bound parameters	`sqltools.bind_parameters()`, driver paramstyle	THREAT_MODEL.md
Single-statement and read-only guard (regex, dialect-aware, fuzz-tested)	`sqltools.validate_sql`	THREAT_MODEL.md
Read-only database sessions	PostgreSQL, MySQL, ClickHouse and SQLite drivers	THREAT_MODEL.md
`allowed_tables` (sqlglot)	`tableguard.py`	THREAT_MODEL.md
Column masking by lineage, failing closed	Result path, cache key, exports	ADR 0005
DuckDB file access locked to `allowed_paths`	DuckDB connection configuration	THREAT_MODEL.md
Rate limits, quotas, concurrency, time and row limits	Each front door; Redis across instances	ADR 0011
Agent writes need a named person's approval, bound to caller, version and arguments	MCP server	ADR 0008
Passwords encrypted at rest (Fernet), rotatable	Store; `queryapigate rekey`	THREAT_MODEL.md
Secret-reference allow-lists	Checked when a reference is saved and when it is used	ADR 0015
Tamper-evident audit log (HMAC chain), streamed to a SIEM	Store transaction; delivery cursor and lease	ADR 0012, ADR 0019
Security headers (nosniff, `no-store`, CSP, `no-referrer`, optional HSTS); SRI on `/docs`	Every response	ADR 0010
CORS off unless origins are set	`cors.py`	THREAT_MODEL.md
`/metrics` behind a bearer token	Main and events servers; on by default in the chart	DEPLOYMENT.md

CONTROL	WHERE ENFORCED	REFERENCE
FIPS-approved algorithms only (test-enforced)	Code; validated module comes from the platform	DATA_PROTECTION.md
TLS termination	Customer's reverse proxy or ingress	DEPLOYMENT.md

QueryAPIGate does not defend against these, and the deployer has to cover them:

- **Administrators' data reach.** An owner, admin or developer can run any SQL on every connection. The database account behind each connection is the real limit.
- **Gaps in the SQL guard.** The guard is defence in depth, not a substitute for least-privileged database grants.
- **Network exposure.** Network perimeter controls are the deployer's.

9. Supply chain

From 0.31, every image (both variants, multi-arch) and the Helm chart is signed keyless with cosign, by digest, by the repository's `docker.yml` release workflow on the release tag. The signature is recorded in Sigstore's public log. Each release is signed in two forms: the Sigstore bundle and the classic `.sig` tag, which admission controllers need. Releases before 0.31 are not signed.

- **Provenance and SBOM:**
  - images carry SLSA provenance, an SPDX SBOM and a GitHub artifact attestation naming the commit;
  - the chart has a GitHub attestation;
  - PyPI releases use trusted publishing, with PEP 740 attestations.
- **Pinned builds:**
  - base images are pinned by digest;
  - every GitHub Action is pinned by commit SHA, and a test refuses a `uses:` without one;
  - downloaded tools are checked against their published checksums;
  - Dependabot proposes updates as reviewed commits.
- **Verifying:** `cosign verify` with the workflow identity and the GitHub OIDC issuer, `gh attestation verify`, the SBOM, and an admission policy that admits only signed images. Sigstore policy-controller and Kyverno are covered. Guide 59 walks through each one. SECURITY.md has the summary.

A signature shows which workflow and tag built a release. It does not show that the code is correct, and builds are not reproducible byte for byte (ADR 0018).

10. Stable and experimental features at 0.34.2

Stable features are under the compatibility promise: they change only by addition, and are renamed or removed only through the deprecation process. Experimental features are outside it.

STATUS	FEATURE	SINCE / REASON	REFERENCE
Stable	PostgreSQL, MySQL, SQLite, DuckDB and ClickHouse connections	Tier 1	README
Stable	Per-key concurrency, time and row limits	0.24	ADR 0011
Stable	Audit log hash chain and `audit verify`	0.25	ADR 0012
Stable	Access log without query strings; `history forget`	0.29	ADR 0016
Stable	Helm chart	0.30	DEPLOYMENT.md
Stable	OAuth for the MCP server, Console SSO, MCP write tools, usage quotas, configuration as code, descriptions, agent usage	0.30	ADR 0017
Stable	Signed releases	0.31	ADR 0018
Stable	Audit log streamed to a SIEM	0.32	ADR 0019
Stable	Live events; `${VAR}`, `${file:...}`, `${vault:...}` and Vault database credentials	0.34	ADR 0021
Experimental	H2, generic JDBC and MongoDB connections	`allowed_tables` doesn't work on them; gaps per type	ADR 0017
Experimental	Tracing with OpenTelemetry	Span conventions still settling. What never goes into a span is already a promise.	ADR 0010
Experimental	Secrets and credentials from AWS (`${aws-sm:...}`, RDS IAM)	Tested against a stand-in, not AWS	ADR 0021